

Paralinear distance and its algorithm for hierarchical clustering of high-dimensional discrete variables

Shuai Wang^{a,1}, Lizhu Hao^{a,1}, Xiaofei Wang^{a,*}, Jianhua Guo^{b,*}

^a KLAS and School of Mathematics and Statistics, Northeast Normal University, Changchun, China

^b School of Mathematics and Statistics, Beijing Technology and Business University, Beijing, China

ARTICLE INFO

Keywords:

Paralinear distance

Latent tree model

Variable clustering

ABSTRACT

Variable clustering is an important tool for mining association rules and explaining the latent mechanisms responsible for generating data. In this work, we aim to study the hierarchical variable clustering algorithm based on the paralinear distance between discrete variables. Firstly, we study the paralinear distance with the multinomial distribution, and point out that any distance with additivity on the graphical tree model has a unique form on the paralinear distance. And then, we suggest a novel hierarchical clustering algorithm, which can determine the local relationships of observed variables as sibling groups and singletons in each level, where the hierarchical structures are indicated between the levels. Furthermore, we show the probably approximately correct (PAC) property of the algorithm, and find out that its sample complexity is sensitive to the diameter of the tree. Finally, by using GPU computation, we demonstrate our discoveries and the applications of our learning algorithms through large-scale experiments on both synthetic and real-world data. Extensive empirical results show that the proposed method is efficient for discovering local structures and latent information.

1. Introduction

The analysis of real-world data poses a significant challenge for both the Statistics and Artificial Intelligence communities due to its intricate underlying mechanisms. Recent years have witnessed a surge of research dedicated to detecting relationships between observed variables by employing diverse probabilistic models [8,29,20,21]. Among these models, hierarchical latent variable models have gained increasing popularity for their ability to provide a flexible explanation of the data. In particular, hierarchical latent class models, also known as latent tree models [30,18], utilize tree structures to intuitively describe relationships among variables. By incorporating unobserved or latent variables, these models effectively capture the inherent diversity of real-world data. Furthermore, the versatility of such models extends to various applications, including probabilistic inference [26], classification [27], multidimensional clustering [4,16], and topic detection [3].

Those algorithms for the structural learning of latent tree models can be roughly divided into two groups: scoring metrics [16,3] or additive distances [5,24,31]. The algorithms in the first group aim at searching the latent tree models that are optimal according to a scoring metric. Though suffering from the expensive computation, they can do the structural learning and parameter estimation at the same time. The algorithms in the second group originate from phylogenetic tree reconstruction [10,22]. The algorithm in [5] has the

* Corresponding author.

E-mail addresses: wangxf341@nenu.edu.cn (X. Wang), jhguo@btbu.edu.cn (J. Guo).

¹ These authors contributed equally to this work.

probably approximately correct property. However, it restricts discrete variables to have the same cardinality, and its consistency is built on the so-called symmetric discrete distribution rather than the conventional multinomial distribution. A further extension work [24] allows observed variables to have various cardinalities. The correct recovery rate of its algorithm has a theoretical guarantee for the multinomial distribution.

For high-dimensional data, the building of a latent tree model usually requires a huge number of latent variables and the construction of a deep tree structure. Though the theoretical consistency guarantees the recovery of the models, three problems must be addressed as the diameter of the tree increases. First, latent variables close to the root usually fail to provide an intuitive and valid explanation of the overall model. Second, a latent tree with a larger diameter makes it harder to give an exact recovery for the structure considering the demand on the sample size. Lastly, the nested learning scheme may lead to the accumulation of errors in the structural learning of the whole model. To avoid these problems, we propose an agglomerative learning algorithm for hierarchical clustering. Instead of recovering the whole tree structure, our proposed method aims to hierarchically mine local structures and abstract feature variables for further learning. Moreover, our agglomerative learning algorithm does not suffer from the accumulation of errors between levels.

The present work makes three main contributions. Firstly, we study the parilinear distance between discrete variables with the multinomial distribution, and show that any distance with additivity on the graphical tree model has a unique form on the parilinear distance. This study provides a further understanding of the form of the additive distance, which is closely related to the latent tree and the phylogenetic tree. Secondly, we suggest a hierarchical clustering algorithm that does not suffer from the accumulation of errors between levels, and provide the probably approximately correct (PAC) property of our algorithm. We find that the sample complexity of the algorithm is sensitive to the diameter of the tree. This theoretical part provides an insight that a deeper latent tree is harder to give an exact recovery for the whole structure. Thirdly, by using GPU computation, we demonstrate our discoveries and the applications of our learning algorithms through large-scale experiments on both synthetic and real-world data. Empirical results show that the proposed method is efficient for discovering local structures and latent information.

The remainder of this paper is organized as follows. We discuss related work in Section 2. In Section 3, we introduce the latent tree model and the parilinear distance. In Section 4, we propose a hierarchical clustering algorithm for recovering the local latent structure of observed variables. In Section 5, we present the results of experiments based on simulations and applications to real-world data. All of the technical details are provided in the Appendix.

2. Related work

Mining hierarchical models of latent variables develops rapidly in the past decades. Specifically, Zhang [30] proposes the latent models with tree structures. This graphical tree model provides an intuitive characterization on variable clustering and facilitates further probabilistic inferences. The original work also provides an algorithm for searching the optimal model from data. In each step, it first generates a number of candidate structures by modifying the structure of the current model. And then it evaluates the candidate models on some scoring criterion and picks the best one to use in the next step of search. This search terminates when the score of the best candidate model is no better than the current one.

This scoring search method is further developed in a series of works [4,16,3] for improving the speed of the search. The algorithm in [4] is applied to the multidimensional clustering of data. It gains efficiency by reducing the number of candidate models examined during search and by reducing the time spent on evaluating the candidate models. The BI algorithm in [16] aims at obtaining flat latent tree models where each latent variable is connected to at least one observed variable. It uses the uni-dimensionality test to detect highly correlated observed variables that should be siblings. One latent variable is then introduced for each sibling cluster and the latent variables are connected by using the Chow-Liu algorithm [6]. It can deal with data sets with hundreds of attributes, and has no restrictions on cardinalities of latent and observed variables. The algorithm in [3] further extends the scoring search technique in [16] for recovering latent trees with thousands of attributes. It proposes a progressive EM strategy for the model construction phase, and improves the efficiency of the parameter estimation in the scoring search when the sample size is very large.

Except these scoring search algorithms, another class of algorithms uses the additive distance to build hierarchical structures for observed variables. It originates from the construction of the phylogenetic tree [10,22], whose internal nodes have the same degree except for the root. A popular algorithm for learning phylogenetic trees is the neighbor-joining method by Saitou and Nei [22]. Its original version proceeds by recursively pairing two variables and introducing a latent variable as the parent of the two variables. Another popular one for recovering phylogenetic trees is the quartet-based distance method. This method constructs a set of quartets for all subsets of four observed variables, and combines those quartets to form a latent tree. In practice, the quartet-based method is usually less accurate than the neighbor-joining method. It can be referred to St. John et al. [14] for further comparisons between the neighbor-joining method and the quartet-based method.

Not limited to the phylogenetic tree, the recursive grouping algorithm [5] is another distance-based method for learning general latent tree graphical models. It adopts the nested scheme and constructs new hidden variables totally depending on the structure generated in the last iteration. Similar to the recursive grouping algorithm, the work [24] also suffers from the accumulation of errors for the nested scheme. Moreover, as discussed in Section 4.2, the sample complexity is sensitive to the diameter of the latent tree. Considering the demand on the sample size, a deeper latent tree is harder to give an exact recovery for the whole structure. In this paper, we hierarchically mine local structures and abstract feature variables for further learning. This reduces the size of our concerned tree. And less sample size is required for an exact recovery of the local structure than the original one. The simulation will be shown in Section 5.2 for the comparison between our algorithm, the neighbor-joining algorithm and the recursive grouping algorithm.

3. The latent tree model and the paralinear distance

Let $G = (W, E)$ be a simple undirected graph, where W is the set of vertices and E is the set of undirected edges. The number of edges incident to a vertex x in G is called the degree of x , which is denoted by $d(x)$. A set L of distinct vertices $[x_0, x_1, \dots, x_m]$ is referred to as a path of length m in G between x_0 and x_m if $(x_{i-1}, x_i) \in E$ for all $i = 1, \dots, m$. Furthermore, if $(x_0, x_m) \in E$, we also refer to L as a cycle in G . If for any two vertices of W there is a path between them in G , then we refer to G as a connected graph. Two disjoint vertex subsets A, B are separated by a vertex subset S in G if for any $x \in A$ and any $y \in B$, every path in G between x and y contains a vertex in S . If (W, E) is a connected simple graph with no cycles, it is referred to as a tree and denoted as T . If a vertex x of T has $d(x) = 1$, it is referred to as a leaf in T . If two leaves x and y in T are adjacent to the same vertex, then we refer to $\{x, y\}$ as a sibling pair in T . A sibling group is a maximal set that any two vertices in it form a sibling pair. If x is a leaf and for any leaf y the pair $\{x, y\}$ is not a sibling pair in T , we call x as a singleton. The diameter of a tree T is the number of nodes on the longest path between two leaves in the tree.

If the vertex set W of G represents a set of random variables, a distribution P is said to be Markov for an undirected graph $G = (W, E)$ if A is conditionally independent of B given S under P whenever A and B are separated by S in G . The graphical model $\mathcal{P}_G$ is the set of all the Markov distributions for $G = (W, E)$. Furthermore, if $\mathcal{P}_G$ is a family of multinomial distributions, it is called a discrete graphical model. Let a graph $T = (W, E)$ be a tree. A graphical model $\mathcal{P}_T$ on T is called as a graphical tree model. Furthermore, the model is referred to as a latent tree model if the leaves of T are observed variables and the internal nodes are unobserved (or latent) variables. We denote the set of observed variables as V and the set of latent variables as H . Thus, the vertex set W of tree T comprises V and H , i.e., $W = V \cup H$. To guarantee that there are no redundant latent variables in the tree model, two identifiability assumptions [5,19] are required:

(A1) each latent variable has at least three neighbors, and

(A2) any pair of variables in the tree model are neither perfectly dependent nor independent.

We define the correlation between discrete variables x and y with the same cardinality k by

$$\rho_{xy} = \frac{\det(P_{xy})}{\sqrt{\det(P_{xx})\det(P_{yy})}}, \quad (1)$$

where P_{xy} is the joint probability matrix between x and y . For binary variables x and y , the correlation ρ_{xy} is the conventional correlation coefficient of them. To have probably approximately property results, we need another assumption: (A3) any joint probability matrix has full rank k . If assumptions (A2) and (A3) hold, we have that $0 < |\rho_{xy}| < 1$.

Definition 3.1. The paralinear distance between variables x and y is defined as $d_{xy} = -\log |\rho_{xy}|$.

The paralinear distance is introduced by [15], and it has the following additive property along the paths in the trees.

Proposition 3.1. For a tree $T = (W, E)$ and a discrete graphical tree model $\mathcal{T}$ on T , if $[x_0 = x, x_1, \dots, x_m = y]$ is a path in T between two variables $x, y \in W$, then the paralinear distance between x and y has the following additive property along the path:

$$d_{xy} = \sum_{i=0}^{m-1} d_{x_i x_{i+1}}.$$

The additive property is implied by the fact that the matrix multiplication $P_{xy} P_{yy}^{-1} P_{yz}$ is equal to P_{xz} when it holds the conditional independence of x and z given y . Furthermore, additive distances for this discrete case demand a homomorphic mapping $f(\bullet) : GL_k(\mathbb{R}) \mapsto \mathbb{R} \setminus \{0\}$, where k is the cardinality of discrete variable and $GL_k(\mathbb{R})$ is the group of $k \times k$ invertible matrices of real numbers. The following theorem shows that if function $f(\bullet)$ is a non-zero polynomial, additive distances for the discrete case have a unique form.

Theorem 3.2. Let function $f(\bullet) : GL_k(\mathbb{R}) \mapsto \mathbb{R} \setminus \{0\}$ be a non-zero polynomial homomorphic mapping. If an additive distance for a discrete graphical tree model $\mathcal{T}$ on T is defined by $-\log |f^\gamma(P_{xx}^{-1/2} P_{xy} P_{yy}^{-1/2})|$ for some $\gamma > 0$, this distance has the form $-s \log |\det(P_{xx}^{-1/2} P_{xy} P_{yy}^{-1/2})|$ where s is some non-negative real number.

Notice that the function f is restricted to be a non-zero polynomial. Following from some techniques in computational algebraic geometry [7] and classic groups [28], we prove in the Appendix A that if further f is a homomorphic mapping from $GL_k(\mathbb{R}) \mapsto \mathbb{R} \setminus \{0\}$, then $f(\mathbf{A}) = \det^g(\mathbf{A})$ for $\mathbf{A} \in GL_k(\mathbb{R})$, where g is a non-negative integral exponent. So it concludes Theorem 3.2 that $-\log |f^\gamma(\bullet)| = -s \log |\det(\bullet)|$ where $s = g\gamma$. According to Theorem 3.2, an additive distance acting on joint probability matrices from a large class of distances has a unique form of $-s \log |\det(\bullet)|$ where s is some non-negative real number. The paralinear distance d_{xy} used in this paper is defined for the case that $s = 1$.

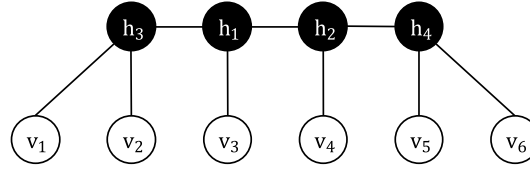


Fig. 1. Simplified illustration of Proposition 3.1.

4. Hierarchical clustering for high-dimensional discrete variables

In this section, we suggest an agglomerative learning algorithm for hierarchical clustering, which can recover the local latent structure of observed variables. Furthermore, we provide the probably approximately correct property and the sample complexity for the algorithm.

4.1. Agglomerative learning for hierarchical clustering

Based on the additivity of paralinear distances, we can assess the sibling pair relationship for any two observed variables. The sufficient and necessary condition [5] for this assessment is given in the following.

Proposition 4.1. Assume that all of the observed variables V are the leaves of a latent tree model. Two observed variables, x and y , are a sibling pair if and only if $\Phi_{xyz} \triangleq d_{xz} - d_{yz}$ is constant for any observed variable $z \in V \setminus \{x, y\}$.

Next, we present an example to illustrate how this Proposition works. Assume that the underlying latent tree has a structure shown in Fig. 1. Variables $v_1, \dots, v_6$ are observed and variables $h_1, \dots, h_4$ are latent. Observed variables v_1 and v_2 form a sibling pair, which is also the case for variables v_5 and v_6 . Let observed variables v_1 and v_2 be fixed, the value of $\Phi_{v_1 v_2 w}$ does not change for $w = v_3, v_4, v_5, v_6$. Let variables v_1 and v_3 be fixed, we have that $\Phi_{v_1 v_3 v_2} \neq \Phi_{v_1 v_3 v_4}$ since $d_{h_1 h_3} > 0$ from assumption (A2). As shown in Section 4.2, the paralinear distance difference Φ_{xyz} has an empirical estimation $\hat{\Phi}_{xyz} = \hat{d}_{xz} - \hat{d}_{yz}$ for $x, y, z \in V$, where the estimation $\hat{d}_{xy}$ can be computed by the empirical probability matrix $\hat{P}_{xy} = (\frac{1}{N} \sum_{i=1}^N \mathbb{I}(x^{(i)} = x_s, y^{(i)} = y_t))_{1 \leq s \leq k, 1 \leq t \leq k}$.

Algorithm 1: Agglomerative Learning for Hierarchical Clustering (A-Learning).

Input : Observed variables V with their order of importance;
 Paralinear distances d_{xy} for any $x, y \in V$;
 The number of levels L considered in the hierarchical modeling;

Output: Sibling groups and singletons in each level;

- 1 Compute $\Phi_{xyz} = d_{xz} - d_{yz}$ for any three variables $x, y, z \in V$;
- 2 Let $X = V$ and $l = 1$;
- 3 **for** $l \leq L$ **do**
- 4 **while** $x, y \in X$ **do**
- 5 **if** Φ_{xyz} is constant for any $z \in X \setminus \{x, y\}$ **then**
- 6 $\{x, y\}$ is a sibling pair.
- 7 **end**
- 8 **end**
- 9 Denote sibling groups by $\{\Pi_l\}_{l=1}^T$ and denote singletons $X \setminus \bigcup_{l=1}^T \Pi_l$ by S ;
- 10 Use the order of importance to select representatives R from groups;
- 11 Let $X = R \cup S$ and $l = l + 1$;
- 12 **end**

As shown in Proposition 4.1, for any two observed variables x and y , we can test their sibling relationship by considering the value of Φ_{xyz} . Thus, we can design the Agglomerative Learning (A-Learning) algorithm for hierarchical clustering of observed variables. The A-Learning algorithm begins with observed variables with their order of importance, their paralinear distances and the number of level. In the first level, by using steps 4-9, some variables are merged into sibling groups, and others are kept alone. Those sibling groups $\{\Pi_l\}_{l=1}^T$ and singletons S are output as a local structure in the first level. For further learning in the higher level, we need a criterion for variable importance ordering to select one representative from every sibling group. For supervised data, we can select the criterion basing on the correlation between the response and feature variables. For unsupervised data, we can obtain information regarding the ordered importance of variables in sibling groups through studying the data background. For example, the average TF-IDF [3] provides an ordered importance of terms in a collection of documents for text analysis. In the next level, further relationships can be checked between all of the representatives R together with the singletons S . According to the

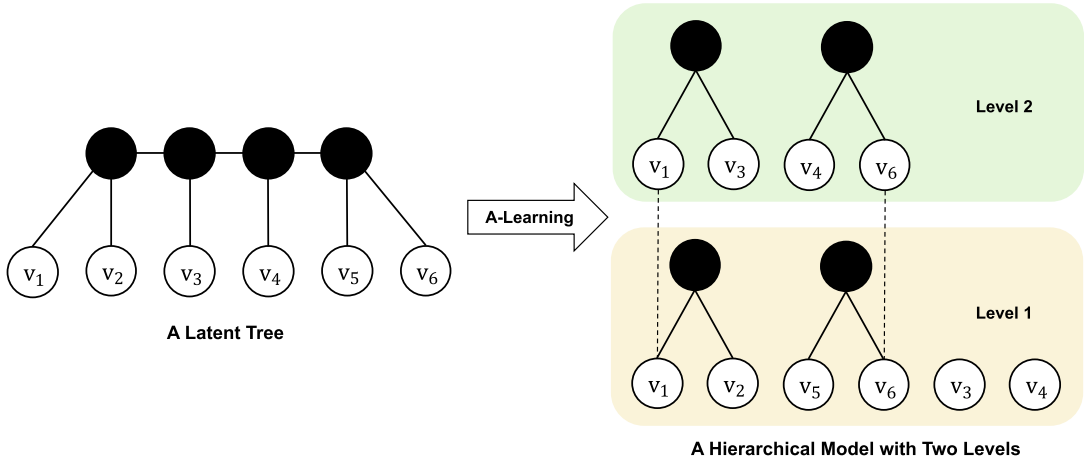


Fig. 2. Flow chart illustrating the mechanism of hierarchical clustering.

collapsibility of graphical models [25,1], the original latent tree model is collapsible onto a subtree model, which can be viewed as a marginalization of the original model, with leaves of representatives R and singletons S . After trimming redundant latent variables, a reduced subtree structure satisfying (A1) is obtained with leaves $R \cup S$. By using steps 4-9 again, the A-Learning algorithm recovers the local structure in the next level.

For learning latent tree models, [5] also presents the Recursive Grouping algorithm, in which the steps 2-3 perform the steps 4-9 in the A-Learning with the addition of detecting the so-called parent-child relationships. They consider the nested scheme and constructs new hidden variables totally depending on the structure generated in the last iteration. Their method may suffer from the accumulation of errors for the nested scheme. Moreover, as discussed in Section 4.2, the sample complexity is sensitive to the diameter of the latent tree. Considering the demand on the sample size, a latent tree with a larger diameter is harder to give an exact recovery for the structure. The simulation study illustrates its performance for deep trees in Section 5.2. To avoid this problem for the learning of the next level, we suggest to only consider the local structure of representatives and singletons in our A-Learning algorithm. This reduces the size of our concerned tree, and does not suffer from the accumulation of errors between levels.

Take Fig. 1 as an example. For $l = 1$, the A-Learning algorithm can generate sibling groups $\{v_1, v_2\}$, $\{v_5, v_6\}$ and singletons $\{v_3, v_4\}$ in the first level. If we select v_1 and v_6 as two representatives of the sibling groups, the original graphical model of Fig. 1 is collapsible onto a subtree whose leaves are $\{v_1, v_3, v_4, v_6\}$. After trimming redundant latent variables of the subtree, variables v_1, v_3, v_4 and v_6 can be viewed as leaves of a latent tree satisfying (A1). For $l = 2$, the A-Learning algorithm can further output two local sibling groups as the second level of hierarchical clustering based on a latent subtree model. As discussed above, by selecting representatives of sibling groups, we can obtain a hierarchical clustering algorithm for observed variables. A flow chart is shown in Fig. 2 to illustrate the mechanism behind the hierarchical clustering.

From Proposition 4.1, the A-Learning algorithm for hierarchical clustering can correctly output those true local structures with $O(Lp^3)$, where p is the number of observed variables and L is the number of levels considered in the hierarchical modeling.

Theorem 4.1. Assume that all of the observed variables are the leaves of a latent tree model. If the joint probability matrices P_{xy} , $x, y \in V$ are available, then the A-Learning algorithm for hierarchical clustering outputs the true sibling groups and singletons in each level within the time $O(Lp^3)$, where p is the number of observed variables in V and L is the number of levels considered in the hierarchical modeling.

In the next subsection, we will give the probably approximately correct (PAC) property for our sample-based A-Learning algorithm, which illustrates the algorithm can correctly recover true local structures with a high probability when the sample size is large enough.

4.2. PAC property of A-Learning

To apply the A-Learning algorithm to data, we need to use the empirical estimation $\hat{P}_{xy}$ of the joint probability matrix P_{xy} for any two observed variables $x, y \in V$. We compute the empirical estimation matrix $\hat{P}_{xy} = (\frac{1}{N} \sum_{i=1}^N \mathbb{I}(x^{(i)} = x_s, y^{(i)} = y_t))_{1 \leq s \leq k, 1 \leq t \leq k}$, where N is the sample size. Furthermore, the empirical estimation $\hat{d}_{xy}$ of the parilinear distance d_{xy} can be obtained from $\hat{P}_{xy}$ and the empirical estimation $\hat{\Phi}_{xyz}$ of the parilinear distance difference Φ_{xyz} can be computed by $\hat{\Phi}_{xyz} = \hat{d}_{xz} - \hat{d}_{yz}$ for $x, y, z \in V$.

Here we provide an informal intuitive proof on the PAC property of our algorithms. To judge the relationships between two variables x and y , we only need to check whether Φ_{xyz} is equal to some constant or not for any $z \in V \setminus \{x, y\}$. In the sample-based A-Learning algorithm, we use the difference $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}|$ to determine the equality of Φ_{xyz} and Φ_{xyw} . Thus, we need a prescribed threshold $\epsilon > 0$ such that $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}| < \epsilon$ if and only if $\Phi_{xyz} = \Phi_{xyw}$. Introduce a lower bound notation $\phi_{\min} = \min\{|\Phi_{xyz} - \Phi_{xyw}| :$

$\Phi_{xyz} \neq \Phi_{xyw}, x, y, z, w \in V\}$. It can be checked that if the inequality $|(\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}) - (\Phi_{xyz} - \Phi_{xyw})| < \epsilon$ holds for some $\epsilon < \frac{1}{2}\phi_{min}$, thus $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}| < \epsilon$ if and only if $\Phi_{xyz} = \Phi_{xyw}$. Therefore, if the event $\{ |(\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}) - (\Phi_{xyz} - \Phi_{xyw})| < \epsilon \}$ occurs for any $x, y, z, w \in V$ happens with a high probability when the sample size N is sufficiently large, we can learn the true structure in each level with a high probability from the sample-based A-Learning algorithm.

The strict description of the PAC property of the A-Learning algorithm is shown in the following Theorem 4.2 (2). We denote σ_{min} as the minimum of the non-zero singular values of all the joint probability matrices for the observed variables, i.e., $\sigma_{min} = \min_{x,y \in V} \sigma_k(P_{xy})$, where k is the number of states for the variables.

Theorem 4.2. Let $\eta \in (0, 1)$. Assume that the A-Learning algorithm is provided with N independent samples from the distribution of discrete observed variables in the set V . Thus

- (1) $\Pr(\text{The A-Learning algorithm outputs the true local structures as } N \rightarrow \infty) = 1$
- (2) there exists a threshold $\epsilon > 0$ such that if the sample size N satisfies

$$\frac{\sqrt{t_0} + 1}{\sqrt{N}} < \frac{\epsilon \sigma_{min}}{16k}, \quad (2)$$

where $t_0 = -\log \frac{\eta}{p^2 k}$, then the A-Learning algorithm returns the true local structure with a probability of at least $1 - \eta$.

Theorem 4.2 (1) shows that with probability one the A-Learning algorithm outputs the true local structures as $N \rightarrow \infty$. Actually, this result can be obtained from Theorem 4.1 and the strong law of large numbers for the empirical estimation matrix. Theorem 4.2 (2) provides the probably approximately correct (PAC) property of the A-Learning algorithm. Its proof on observed variables with the same cardinality can be built as a special case of the work [24], so we omit it in this paper. The Recursive Grouping algorithm [5] also considers a proof for the symmetric discrete distribution, while our result here works for the multinomial distribution.

The sample complexity (2) of the A-Learning algorithm actually depends on two unknown intrinsic parameters ϕ_{min} and σ_{min} . The unknown parameter ϕ_{min} determines the upper bound of the prescribed threshold ϵ , which is used to assess whether $\Phi_{xyz} = \Phi_{xyw}$ or not. If $\Phi_{xyz} \neq \Phi_{xyw}$, there are two latent variable h_1 and h_2 such that $|\Phi_{xyz} - \Phi_{xyw}| = 2d_{h_1 h_2} > 0$. So $\phi_{min} = 2d_{h_1 h_2}$ for some $h_1, h_2 \in H$. The minimum singular value σ_{min} is closely related to the diameter of the latent tree. Roughly, a longer path between two observed variables may generate a smaller singular value. Let $[x_0 = x, x_1, \dots, x_m = y]$ be a path between observed variables x and

y . The joint probability matrix P_{xy} can be decomposed into $P_{x_0|x_1} P_{x_1|x_2} \dots P_{x_{m-1}|x_m} P_{x_m x_m}$. Thus $\det(P_{xy}) = \det(P_{yy}) \prod_{i=0}^{m-1} \det(P_{x_i|x_{i+1}})$. By Assumption (A2), it holds that $|\det(P_{x_i|x_{i+1}})| < 1$ for any i . Furthermore, the smallest singular value $\sigma_k(P_{xy}) < 1$ and it tends to be small when the length m is large. The requirement on N strongly depends on the value of σ_{min} . From (2), we need the sample size N is larger than C/σ_{min}^2 where C is a constant given the model and η . So the sample complexity is sensitive to the diameter of the tree since a large diameter can generate a very small σ_{min} .

Though we can theoretically recover the true local structure with a high probability, a very huge number of samples are required by (2) when the diameter of a latent tree is large. In the following simulation experiment, we will see that it is tough to recover sibling groups for a deep latent tree, but it is feasible to learn the local structure for a broad latent tree even if high dimensional data is involved. Moreover, the A-Learning approach demonstrates a higher rate of successful recovery for sibling groups compared to the latent tree learning algorithm [5]. This discrepancy could stem from the fact that the A-Learning algorithm involves fewer assessments of the difference $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}|$ in the context of learning the initial level, in contrast to the comprehensive tree structure learning process. Since latent variables close to the root usually fail to provide an intuitive and valid explanation of the overall model, we choose a small number of levels L in the hierarchical clustering. In general, we are interested in the case where $L = 2$ or 3 because it usually provides sufficient clustering information.

5. Numerical experiment

In this section, we presented numerical experiments for simulated and real data. In subsection 5.1, we introduced the implementation details for the algorithms. In subsection 5.2, we demonstrated the consistency of the A-Learning algorithm based on simulated data, and compared it with several clustering algorithms. In subsections 5.3 and 5.4, we showed the results of experiments using two types of real data: New York Times data and Chinese text data on the Changchun Mayor Public Hotline. Parts of the numerical experiments were executed on a desktop computer with an Intel i9 CPU 3.5 GHz processor and 128 GB memory. For a large-scale simulation with 10k variables with 1000k sample size, we adopted the GPU paralleling computation on GeForce RTX 2080 Ti by using CUDA 11.0.3.

5.1. Implementation details

As mentioned in the last section, the basic sibling pair structure is determined using a prescribed threshold $\epsilon > 0$. In particular, if $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}| < \epsilon$ for all $z, w \in V \setminus \{x, y\}$, the pair $\{x, y\}$ is set to a sibling pair. However, if there are two observed variables z, w such that $|\hat{\Phi}_{xyz} - \hat{\Phi}_{xyw}| \geq \epsilon$, the pair $\{x, y\}$ is not a sibling pair. We refer to ϵ as a structure threshold. It is apparent that when we increase this structure threshold, more observed variables tend to belong to the same sibling group and less become singletons.

Theorem 4.2 provides the PAC property for the A-Learning algorithm, but we still need the condition that $\epsilon < \min\{\frac{1}{2}\phi_{\min}, 1\}$, where the intrinsic parameter $\phi_{\min}$ is defined by $\min\{|\Phi_{xyz} - \Phi_{xyw}| : \Phi_{xyz} \neq \Phi_{xyw}, x, y, z, w \in V\}$. For real data, it is difficult to obtain any information about this parameter. Moreover, the inequality (2)) is difficult to verify because of the other intrinsic parameter $\sigma_{\min}$. Thus, we actually have no a priori knowledge of this structure threshold ϵ , or even its order of magnitude.

In the simulation study, we introduced another tolerance threshold λ as a rough ratio to represent the singletons in all the leaves. This tolerance threshold was a criterion to stop the increase in the structure threshold ϵ . We started the threshold ϵ from a fixed value and it increased with a fixed stepsize. The number of singletons obtained by the A-Learning decreased as the structure threshold increased. We did not stop increasing the threshold ϵ until the number of singletons obtained by the A-Learning algorithm exceeded $n\lambda$, where n is the number of leaves. The introduction of the tolerance threshold λ avoided the difficulty of choosing the structure threshold ϵ . Different values of λ could yield different local structures or local models. Thus, the tolerance threshold actually determines the number of clusters of observed variables. We know that λ can vary in $[0, 1)$, but choosing a suitable sample-based value for λ is still a challenging problem, which is similar to estimating the number of clusters in a data set [23]. We do not discuss this issue further in the present study.

5.2. Simulation study

In this subsection, we mainly illustrated the probably approximately correct (PAC) property of the A-Learning algorithm for local structural learning. We first ran the A-Learning algorithm on two toy tree models, and compared it with the K-medoids method. Furthermore, we showed its performances for the depth and breadth tree structures, and compared it with two algorithms based on additive distances.

We designed the two simple tree structures shown in Fig. 3 (a) and (b) to illustrate the PAC property of the A-Learning algorithm. For each structure, we built latent tree models of binary discrete variables. We randomly generated 100 groups of parameters which were conditional probability matrices in the tree model. The data was further produced for each group of parameters. The sample sizes varied among 1k, 2k, 3k, 5k, 7k, 10k, 15k, 30k, and 100k. The left panel of Fig. 3 (c) displays the average cluster recovery rates [13] of the A-Learning algorithm. As the sample size increased, those cluster recovery rates also exhibited an upward trend, which gave a support on the PAC property of our A-Learning algorithm.

We also conducted a comparative analysis of the A-Learning algorithm with two other clustering methods: a general distance-based algorithm, K-medoids [12], and a rank-based latent causal discovery (RLCD) method [9,13], specifically applied for clustering observed variables. For the K-medoids, paralinear distances were employed, and the number of clusters was set to four and three for Tree 1 and Tree 2, respectively. The performance of the algorithms was assessed by examining the disparity between the learned and true local relationships, utilizing both the cluster recovery rate [13] and the structural Hamming distance (SHD), as illustrated in Fig. 3 (c). Broadly speaking, both the A-Learning algorithm and the RLCD method had improved performance with increasing sample size. In comparison to the K-medoids, these two algorithms performed notably well for Tree 2, with the A-Learning algorithm demonstrating the lowest SHD at each sample size. While the K-medoids achieved a higher cluster recovery rate for Tree 1 compared to the other algorithms, the A-Learning algorithm had low SHDs for Tree 1. Despite Tree 1 being seemingly simpler than Tree 2, the larger diameter of Tree 1 and the presence of extra singletons increased the challenge of structure learning for both the A-Learning and RLCD methods, necessitating larger sample sizes for achieving high cluster recovery rates and low SHDs.

We further designed two kinds of trees based on the depth and breadth to illustrate the performance of the A-Learning algorithm, and compared the A-Learning algorithm with two other additive distance algorithms: the Recursive Grouping (RG) algorithm [5] and the Neighbor Joining (NJ) algorithm [22]. As shown in Fig. 4 (a) and (b), the depth tree structure has a larger diameter as the number of variables increases, and the breadth tree structure keeps a fixed diameter as the number of variables increases. For the breadth tree, the diameter was fixed to five.

Two experiments were done for those two structures of each sibling group formed by three leaves. We built latent tree models of binary discrete variables, and randomly generated 100 groups of parameters of conditional probability matrices. The data was further produced for each group of parameters. The sample sizes varied among 10k, 30k, 50k, 70k and 100k. The average rates are shown in Fig. 5 for correctly recovering the true local relationships between the observed variables. The left panel of Fig. 5 shows that the correct recovery rate for the depth tree dropped heavily as the number of observed variables increased. The A-Learning algorithm performs better than the RG algorithm and the NJ algorithm when the sample size is greater than 50k. Compared to the depth trees, the right panel of Fig. 5 illustrates that the A-Learning has a higher correct recovery rate for the breadth trees. And the A-Learning gets a better performance for 15 observed variables than the RG and the NJ. It also outperforms the RG for 21 observed variables. The reason may lie in that the nested learning scheme in the RG algorithms leads to the accumulation of errors for the structural learning of the whole model.

In the last part of the simulated experiments, we considered two kinds of large-scale breadth trees with 1000, 2000, 5000, 10000 observed variables. The first kind of breadth tree (5_Breadth Tree) has 201, 401, 1001, 2001 latent variables, and each sibling group in this tree consists of five leaves. The second one (10_Breadth Tree) has 101, 201, 501, 1001 latent variables, and each sibling group consists of ten leaves. We randomly generated 100 groups of parameters and data for each structure. The data sample sizes varied among 10k, 100k, 300k, 500k, 700k and 1000k. To complete this large-scale simulation, we adopted the GPU paralleling computation on NVIDIA GeForce RTX 2080 Ti by using CUDA. As shown in Fig. 6, the correct recovery went up as the sample size increased. Roughly, our A-Learning algorithm performed better on the 10_Breadth Tree than the 5_Breadth Tree. Though the 10_Breadth Tree has larger sibling groups than the 5_Breadth Tree, the 10_Breadth Tree has fewer latent variables.

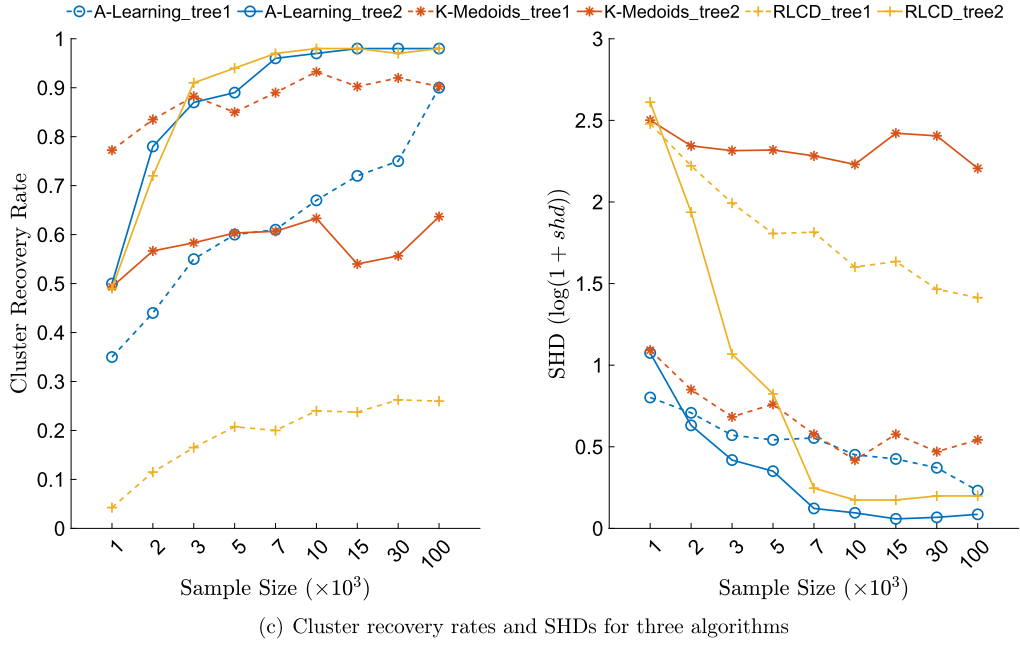
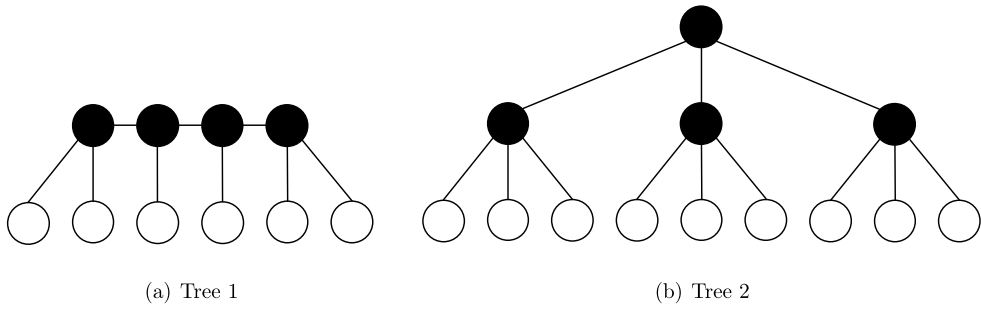


Fig. 3. Performance of the A-Learning in local relationship recovery.

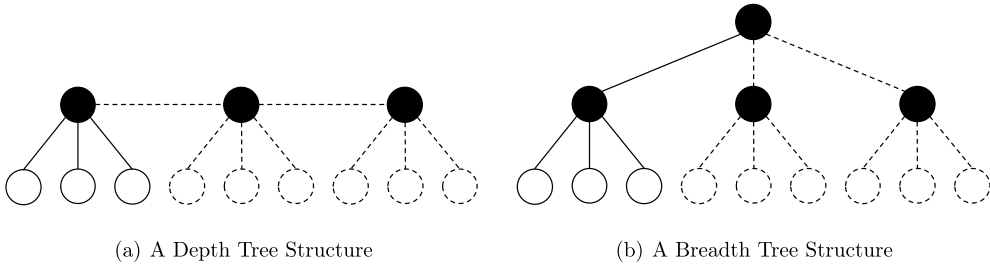


Fig. 4. Two kinds of trees on the depth and breadth.

In the following subsections, we performed using our algorithms based on real-world data sets. The probability distributions that govern these data sets might not satisfy the assumptions required for consistent learning of the latent tree models. However, our experiments demonstrated that our agglomerative algorithm can still be useful for mining the latent relevances behind variables. Those relevances, reflecting topics in text data, can be viewed as the possible meaning of latent variables related to sibling groups of observed variables.

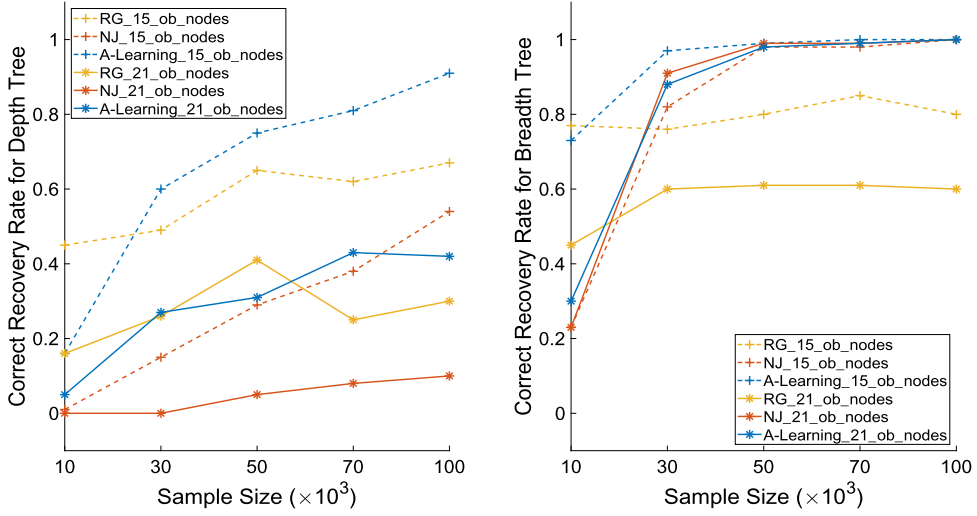


Fig. 5. Correct recovery rates on depth and breadth-based trees.

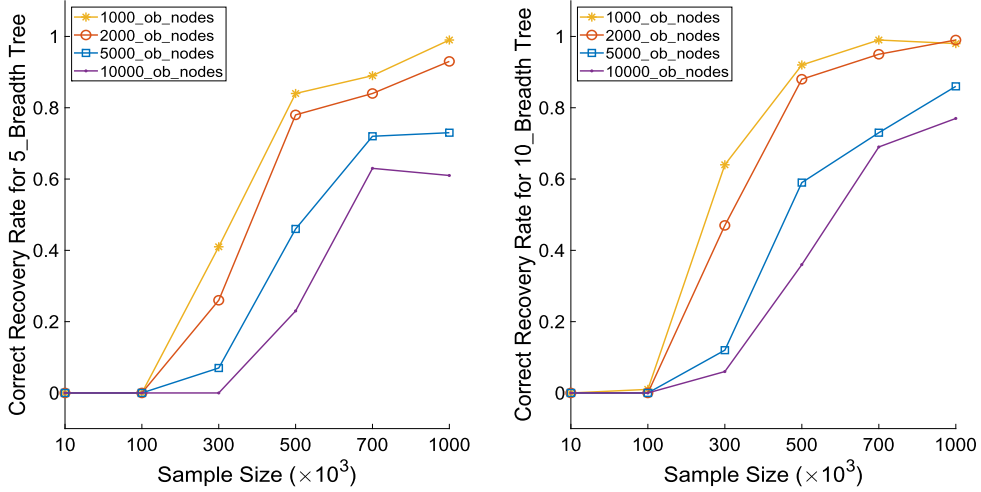


Fig. 6. Correct recovery rates for two varieties of large-scale breadth trees.

5.3. English text application

We next applied the A-Learning to the New York Times dataset² consisting of 300,000 articles. A vocabulary of 10,000 English keywords was selected using average TF-IDF [3] for the analysis. The average TF-IDF of a term t in a collection of documents D is defined as follows:

$$\text{tf-idf}(t, D) = \frac{\sum_{d \in D} \text{tf}(t, d) \text{idf}(t, D)}{|D|}, \quad (3)$$

where $|\cdot|$ stands for the cardinality of a set, $\text{tf}(t, d)$ is the term frequency of t in document d , and $\text{idf}(t, D) = \log(|D| / |\{d \in D : t \in d\}|)$ is the inverse document frequency of t in the document collection D . Using the A-Learning algorithm, we obtained 6318 singletons and 1139 sibling groups. 738 groups have two keywords, 194 groups have three keywords, 85 groups have four keywords, and 122 groups have more than four keywords. The largest group consists of 118 keywords. In Fig. 7, we show the bar chart for the size of sibling groups and the first seven sibling groups. The fourth sibling group, consisting of 26 keywords, is closely related to a topic on film. The sixth sibling group consists of four keywords: attack, terrorist, terrorism, terror. It reflects a topic on terrorism.

² <http://archive.ics.uci.edu/ml/datasets/Bag+of+Words>.

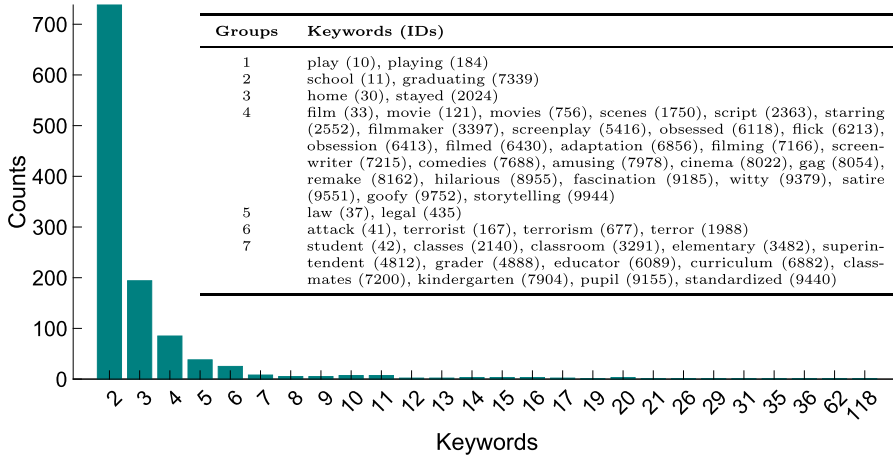


Fig. 7. Sibling group sizes and initial group data.

Table 1

Average topic coherence scores and average paralignear distances.

Model	Coher ₂	Coher ₃	Coher ₄	DisWithin	DisBetween	DisWithin/DisBetween
LDA	*	*	-10.75	2.48	4.39	0.56
HLTA	-1.66	-6.93	-16.34	2.82	5.51	0.51
A-Learning	-2.41	-7.55	-15.59	2.22	5.78	0.38

To assess the topic quality of sibling groups, we considered the topic coherence score [17], whose intuition lies in that keywords in a good topic should tend to co-occur in the documents. Suppose a topic t is characterized using a list $G^{(t)} = \{w_1^{(t)}, \dots, w_{M_t}^{(t)}\}$ of M_t keywords. The coherence score of topic t is given by:

$$\text{Coherence}(G^{(t)}) = \sum_{i=2}^{M_t} \sum_{j=1}^{i-1} \log \frac{D(w_i^{(t)}, w_j^{(t)}) + 1}{D(w_j^{(t)})} \quad (4)$$

where $D(w_i^{(t)})$ is the number of documents containing keyword $w_i^{(t)}$, and $D(w_i^{(t)}, w_j^{(t)})$ is the number of documents containing both $w_i^{(t)}$ and $w_j^{(t)}$. With M_t fixed, a higher coherence score indicates a better quality topic. For $M_t = 2, 3, 4$, we computed each average topic coherence score for sibling groups of M_t keywords. In addition, we also considered the average paralignear distance within topics and the average paralignear distance between topics. A good topic should have a low average paralignear distance within topics, and a high average paralignear distance between topics. Suppose we have a class of topics $\mathcal{G} = \{G^{(1)}, \dots, G^{(T)}\}$ and $G^{(t)} = \{w_1^{(t)}, \dots, w_{M_t}^{(t)}\}$ for any topic t . The way to compute the average paralignear distances within and between topics is given by:

$$\text{DisWithin} = \frac{1}{T} \sum_{t=1}^T \frac{2}{M_t(M_t - 1)} \sum_{i=2}^{M_t} \sum_{j=1}^{i-1} \text{dis}(w_i^{(t)}, w_j^{(t)}), \quad (5)$$

$$\text{DisBetween} = \frac{2}{T(T-1)} \sum_{t=2}^T \sum_{s=1}^{t-1} \frac{1}{M_t M_s} \sum_{i=1}^{M_t} \sum_{j=1}^{M_s} \text{dis}(w_i^{(t)}, w_j^{(s)}) \quad (6)$$

where $\text{dis}(w_i, w_j)$ is the paralignear distance between keywords w_i and w_j .

Basing on the coherence score and the paralignear distance, we compared the performance of the A-Learning with two other methods: latent Dirichlet allocation (LDA) and hierarchical latent tree analysis (HLTA). The LDA method [2] models the process of document generation for the topic discovery. The HLTA method [3] detects hierarchical topics by using a whole latent tree structure. For the LDA, each topic is a distribution over all the keywords, and the topic is characterized using a few words with high probabilities. In the experiment, we set each topic with four highest ranking (most probable) keywords for the LDA. Considering the computation efficiency of the HLTA, we chose 10,000 samples for its model construction. The comparison result of the three methods is shown in Table 1. The A-Learning outperformed the HLTA and the LDA on the DisWithin, the DisBetween and their ratio. For topics with two or three keywords, the HLTA performed better than the A-Learning on the average topic coherence. For topics with four keywords, the LDA performed best on the average topic coherence.

Table 2
Performances of the A-Learning algorithm for 10k, 20k, 30k keywords.

Keywords	Coher ₂	Coher ₃	Coher ₄	DisWithin	DisBetween	DisWithin/DisBetween
10,000	-2.88	-10.97	-24.19	2.99	7.33	0.41
20,000	-3.01	-11.41	-25.03	3.36	8.15	0.41
30,000	-3.04	-11.15	-24.39	3.82	8.98	0.43

5.4. Chinese text application

The Changchun mayor hotline [11] is an important project led by the local government of Changchun city, which is the capital of Jilin Province in Northeast China. This project provides local residents with the opportunity to call the mayor's office and report various public issues via the "12345" hotline. Each phone call is recorded and converted into a text document in Chinese by an operator, and each document is assigned to a department of the Changchun government. The typical issues reported include local crime, public utility, and problems with transportation.

From 2005 to 2016, a text database with 74,589 distinct Chinese words was built for this Changchun mayor hotline. The whole database consists of a total of 2,401,318 documents assigned to 113 departments. For this database, our A-Learning algorithm can learn sibling groups for 30k keywords within half an hour. We did not compare the A-Learning with the LDA and the HLTa in this part because their computational efficiency is not fit for large databases with millions of high-dimensional samples. Table 2 shows the performances of the A-Learning algorithm on average topic coherence scores and average paralignar distances.

In addition to the large-scale computation on the whole dataset, we also studied the learning results on the top 100 Chinese keywords of the largest correlation (1) with the Public Security Bureau (PSB), which is assigned to a total of 170,705 complaint documents. Using the A-Learning three times, we obtained a hierarchical clustering with three levels. There are 20 sibling groups and 32 singletons in the first level, 7 sibling groups and 19 singletons in the second level, 2 sibling groups and 6 singletons in the third level.

In some cases, a sibling group reflected a specific topic related to the PSB. In the first level, the group {gambling, group gambling} represented a topic related to gambling. Representative variables with the largest correlation in each group and singletons in the first level could form further new sibling groups in the next levels. The group {large breed dog, dog barking, yell, walk dog} in the second level consisted of one representative variable "large breed dog" and three singletons "dog barking, yell, walk dog" in the first level. The group {mahjong parlor, gambling} in the third level consisted of two representative variables "mahjong parlor" and "gambling" in the first level. In the first level, {mahjong parlor, mahjong, play mahjong} and {gambling, group gambling} were two sibling groups. In the third level, the sibling group {mahjong parlor, gambling} illustrated the close relation between {mahjong parlor, mahjong, play mahjong} and {gambling, group gambling}. Thus, the A-Learning algorithm provides a hierarchical clustering method for building the possible underlying relevance between various topics. A detailed discussion of the hierarchical clustering of those 100 Chinese keywords can be found in 'Supplementary A'.

6. Conclusion

In this study, we considered the paralignar distance with the multinomial distribution, and pointed out that any distance with additivity on the graphical tree model has a unique form on the paralignar distance. Moreover, we proposed a novel variable clustering method, which can mine the relationships between observed variables as sibling groups and singletons. We demonstrated that this method has the PAC property based on latent tree models. The potential applications of the proposed method include text mining and the classification of high-dimensional data. Extensive empirical results show that the proposed method is efficient for discovering local structures and latent information.

CRedit authorship contribution statement

Shuai Wang: Conceptualization, Data curation, Writing – original draft. **Lizhu Hao:** Data curation. **Xiaofei Wang:** Conceptualization, Writing – original draft, Writing – review & editing. **Jianhua Guo:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgements

The authors would like to thank the reviewers for their constructive suggestions and comments that improved the quality of this paper. Special thanks are extended to Prof. Kun Zhang's team from CMU for generously sharing their latest code with us. This work is supported by the National Key R&D Program of China (No. 2020YFA0714102), and the National Natural Science Foundation of China (No. 12171076, 12226003).

Appendix A. Proofs of Theorem 3.2

Theorem 3.2 can be concluded from the following proposition, which follows from the technique of classical groups [28]. Here we give a direct proof based on the matrix computation.

Proposition A.1. *If a non-zero polynomial $f(\bullet)$ on $\mathbb{R}^{k \times k}$ satisfies that $f(AB) = f(A)f(B)$ for any two matrices $A, B \in \mathbb{R}^{k \times k}$, this polynomial $f(\bullet)$ must be a power of the determinant on $\mathbb{R}^{k \times k}$.*

Proof. First, consider a special case that $A = I$ is an identity matrix. Since $f(AB) = f(A)f(B)$, thus $f(I) = 1$ or 0. If $f(I) = 0$, thus $f(A) = 0$ for any $A \in \mathbb{R}^{k \times k}$. Since $\mathbb{R}^{k \times k}$ is an infinite field, $f \equiv 0$ by Proposition 5 [7]. It is in contradiction with that f is a non-zero polynomial. So $f(I) = 1$. Similarly, $f(\mathbf{0}) = 1$ or 0 where $\mathbf{0}$ is a zero matrix. If $f(\mathbf{0}) = 1$, thus $f(A)f(\mathbf{0}) = f(\mathbf{0}) = 1$ for any $A \in \mathbb{R}^{k \times k}$. So $f(A) = 1$ for any $A \in \mathbb{R}^{k \times k}$. In the following part we consider $f(\mathbf{0}) = 0$. So there is no monomial of degree zero in polynomial f .

Since f is a non-zero polynomial on $\mathbb{R}^{k \times k}$, $f(\text{diag}(\lambda_1, \dots, \lambda_k))$ is a non-zero polynomial on $\mathbb{R}^k$. Denote A^{ij} as an elementary matrix which permuting the i -th and j -th rows of A . It can be verified that $f(A^{ij}A^{ij}) = f(I) = 1$. Furthermore, $f(\text{diag}(\lambda_1, \dots, \lambda_k)) = f(\text{diag}(\lambda_{s(1)}, \dots, \lambda_{s(k)}))$ for any permutation $(s(1), \dots, s(k))$ of $(1, \dots, k)$. When some $\lambda_i = 0$, it can be obtained that $f(\text{diag}(\lambda_1, \dots, \lambda_k)) \equiv 0$ in the case that $f(\mathbf{0}) = 0$. Actually, the zero matrix $\mathbf{0}$ can be obtained by multiplying $n-1$ permutation matrices of $\text{diag}(\lambda_1, \dots, \lambda_k)$ when some $\lambda_i = 0$. Since $f(\text{diag}(\lambda_1, \dots, \lambda_k)) = f(\text{diag}(\lambda_{s(1)}, \dots, \lambda_{s(k)}))$, we have that $f(\text{diag}(\lambda_1, \dots, \lambda_k)) \equiv 0$ by $f(AB) = f(A)f(B)$.

It can be verified that every monomial in $f(\text{diag}(\lambda_1, \dots, \lambda_k))$ has a factor $\lambda_1 \cdots \lambda_k$. Otherwise, for example, if there is a monomial $c\lambda_1^{m_1} \cdots \lambda_{k-1}^{m_{k-1}}$ in function $f(\text{diag}(\lambda_1, \dots, \lambda_k))$, we can set $\lambda_k = 0$. So $f(\text{diag}(\lambda_1, \dots, \lambda_k)) \equiv 0$. It means that $c\lambda_1^{k_1} \cdots \lambda_{n-1}^{k_{n-1}} \equiv 0$. It is a contradiction. So we can extract a common factor $\lambda_1 \cdots \lambda_k$ in $f(\text{diag}(\lambda_1, \dots, \lambda_k))$ such that

$$f(\text{diag}(\lambda_1, \dots, \lambda_k)) = \lambda_1 \cdots \lambda_k g(\text{diag}(\lambda_1, \dots, \lambda_k))$$

and $g(\text{diag}(\lambda_1, \dots, \lambda_k))$ is a non-zero polynomial on $\mathbb{R}^k$. Since

$$f(\text{diag}(\lambda_1, \dots, \lambda_k)\text{diag}(\mu_1, \dots, \mu_k)) = f(\text{diag}(\lambda_1, \dots, \lambda_k))f(\text{diag}(\mu_1, \dots, \mu_k)),$$

So

$$(\lambda_1 \mu_1) \cdots (\lambda_k \mu_k)(g(\text{diag}(\lambda_1 \mu_1, \dots, \lambda_k \mu_k)) - g(\text{diag}(\lambda_1, \dots, \lambda_k))g(\text{diag}(\mu_1, \dots, \mu_k))) = 0.$$

It means that when $\lambda_1 \cdots \lambda_k \mu_1 \cdots \mu_k \neq 0$,

$$g(\text{diag}(\lambda_1 \mu_1, \dots, \lambda_k \mu_k)) - g(\text{diag}(\lambda_1, \dots, \lambda_k))g(\text{diag}(\mu_1, \dots, \mu_k)) = 0.$$

From the continuity of polynomial,

$$g(\text{diag}(\lambda_1 \mu_1, \dots, \lambda_k \mu_k)) = g(\text{diag}(\lambda_1, \dots, \lambda_k))g(\text{diag}(\mu_1, \dots, \mu_k))$$

for any $\lambda_1, \dots, \lambda_k$ and any $\mu_1, \dots, \mu_k$. By similar discussion, $g(I) = 1$ and $g(\mathbf{0}) = 0$ or 1. When $g(\mathbf{0}) = 1$, $g(\text{diag}(\lambda_1, \dots, \lambda_k)) \equiv 1$ for any $\lambda_1, \dots, \lambda_k$. So $f(\text{diag}(\lambda_1, \dots, \lambda_k)) = \lambda_1 \cdots \lambda_k$. Otherwise, we consider the case that $g(\mathbf{0}) = 0$. As discussed above, we have that $g(\text{diag}(\lambda_1, \dots, \lambda_k)) = g(\text{diag}(\lambda_{s(1)}, \dots, \lambda_{s(k)}))$. Thus $g(\text{diag}(\lambda_1, \dots, \lambda_k)) = 0$ when some $\lambda_i = 0$. So every monomial in $g(\text{diag}(\lambda_1, \dots, \lambda_k))$ should contain a factor $\lambda_1 \cdots \lambda_k$. By recursive computation, we have that

$$f(\text{diag}(\lambda_1, \dots, \lambda_k)) = (\lambda_1 \cdots \lambda_k)^G$$

where G is a non-negative integral exponent.

If A is a diagonal matrix whose diagonal elements all equal θ , we have that $f(A)$ equals θ^G where G is a non-negative integral exponent. For any $A \in \mathbb{R}^{k \times k}$, we choose a matrix B such that $BA = AB = \Delta I$ where Δ is the determinant of A . So $f(A)f(B) = f(\Delta I) = \Delta^G$. Since the determinant Δ is an irreducible polynomial of n^2 variables, the function $f(A)$ is a power Δ^g where g is a non-negative integral exponent. $\square$

Appendix B. Supplementary material

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.ijar.2024.109133>.

References

- [1] S. Asmussen, D. Edwards, Collapsibility and response variables in contingency tables, *Biometrika* 70 (1983) 567–578.
- [2] D.M. Blei, A.Y. Ng, M.I. Jordan, Latent Dirichlet allocation, *J. Mach. Learn. Res.* 3 (2003) 993–1022.
- [3] P.X. Chen, N.L. Zhang, T.F. Liu, Leonard K.M. Poon, Z.R. Chen, F. Khawar, Latent tree models for hierarchical topic detection, *Artif. Intell.* 250 (2017) 105–124.
- [4] T. Chen, N.L. Zhang, T.F. Liu, K.M. Poon, Y. Wang, Model-based multidimensional clustering of categorical data, *Artif. Intell.* 176 (1) (2012) 2246–2269.
- [5] M.J. Choi, V.Y.F. Tan, A. Anandkumar, A.S. Willsky, Learning latent tree graphical models, *J. Mach. Learn. Res.* 12 (2011) 1771–1812.
- [6] C.K. Chow, C.N. Liu, Approximating discrete probability distributions with dependence trees, *IEEE Trans. Inf. Theory* 14 (1968) 462–467.
- [7] D. Cox, J. Little, D. O’Shea, *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*, Springer, 2007.
- [8] K. Deng, Z. Geng, J.S. Liu, Association pattern discovery via theme dictionary models, *J. R. Stat. Soc. Ser. B* 76 (2014) 319–347.
- [9] X. Dong, B. Huang, I. Ng, X. Song, Y. Zheng, S. Jin, R. Legaspi, P. Spirtes, K. Zhang, A versatile causal discovery framework to allow causally-related hidden variables, *arXiv preprint arXiv:2312.11001*, 2023.
- [10] P.L. Erdős, L.A. Székely, M.A. Steel, T.J. Warnow, A few logs suffice to build (almost) all trees: part ii, *Theor. Comput. Sci.* 221 (1999) 153–184.
- [11] G.Y. Guan, J.H. Guo, H.S. Wang, Varying Naïve Bayes models with applications to classification of Chinese text documents, *J. Bus. Econ. Stat.* 32 (2014) 445–456.
- [12] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, second edition, Springer, 2009.
- [13] B. Huang, C. Low, X. Feng, C. Glymour, K. Zhang, Latent hierarchical causal structure discovery with rank constraints, in: *NeurIPS*, 2022.
- [14] K.St. John, T. Warnow, B.M.E. Moret, L. Vawter, Performance study of phylogenetic methods: (unweighted) quartet methods and neighbor-joining, *J. Algorithms* 48 (2003) 173–193.
- [15] J.A. Lake, Reconstructing evolutionary trees from dna and protein sequences: paralogous distances, *Proc. Natl. Acad. Sci.* 91 (1994) 1455–1459.
- [16] T.F. Liu, N.L. Zhang, P.X. Chen, A.H. Liu, L.K.M. Poon, Y. Wang, Greedy learning of latent tree models for multidimensional clustering, *Mach. Learn.* 98 (2015) 301–330.
- [17] D. Mimno, H.M. Wallach, E. Talley, M. Leenders, A. McCallum, Optimizing semantic coherence in topic models, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2011, pp. 262–272.
- [18] R. Mourad, C. Sinoquet, N.L. Zhang, T.F. Liu, P. Leray, A survey on latent tree models and applications, *J. Artif. Intell. Res.* 47 (2013) 157–203.
- [19] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [20] L.K.M. Poon, N.L. Zhang, T.F. Liu, A.H. Liu, Model-based clustering of high-dimensional data: variable selection versus facet determination, *Int. J. Approx. Reason.* 54 (2013) 196–215.
- [21] F. Rodriguez-Sanchez, C. Bielza, P. Larrañaga, Multipartition clustering of mixed data with bayesian networks, *Int. J. Intell. Syst.* 37 (2022) 2188–2218.
- [22] N. Saitou, M. Nei, The neighbor-joining method: a new method for reconstructing phylogenetic trees, *Mol. Biol. Evol.* 4 (1987) 406–425.
- [23] R. Tibshirani, G. Walther, T. Hastie, Estimating the number of clusters in a data set via the gap statistic, *J. R. Stat. Soc. Ser. B* 63 (2001) 411–423.
- [24] X.F. Wang, J.H. Guo, L.Z. Hao, N.L. Zhang, Spectral methods for learning discrete latent tree models, *Stat. Interface* 10 (2017) 677–698.
- [25] X.F. Wang, J.H. Guo, X. He, Finding the minimal set for collapsible graphical models, *Proc. Am. Math. Soc.* 139 (2011) 361–373.
- [26] Y. Wang, N.L. Zhang, T. Chen, Latent tree models and approximate inference in Bayesian networks, *J. Artif. Intell. Res.* 32 (2008) 879–900.
- [27] Yi Wang, Nevin L. Zhang, Tao Chen, Leonard K.M. Poon, Ltc, A latent tree approach to classification, *Int. J. Approx. Reason.* 54 (4) (2013) 560–572.
- [28] H. Weyl, *The Classical Groups: Their Invariants and Representation*, Princeton University Press, 1939.
- [29] X.C. Xie, Z. Geng, Q. Zhao, Decomposition of structural learning about directed acyclic graphs, *Artif. Intell.* 170 (2006) 422–439.
- [30] N.L. Zhang, Hierarchical latent class models for cluster analysis, *J. Mach. Learn. Res.* 5 (2004) 697–723.
- [31] C. Zhou, X.F. Wang, J.H. Guo, Learning mixed latent tree models, *J. Mach. Learn. Res.* 21 (2020) 1–35.